



第11章：复杂推理——思维链 与数学能力



学习目标与主线

01

Prompt 如何从“技巧”走向“可编程/可优化的推理流程”

02

数学推理为何难，怎样用“工具/验证/形式化”补齐？

03

垂直领域怎样把推理做得更可靠、可审计？



Prompt Learning 进阶

推理的两种形态：快答 vs 慢思考



LYCHEE



哈尔滨工业大学(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

快答

- 快答 (Direct / System 1 的类比)：给一个问题，模型几乎“一次生成到底”，直接给答案
- 为什么容易错：没分解/没自检/没回溯
- 类比成人的“直觉反应”：快、便宜、但在复杂题上容易犯低级错

直出 (Direct) 为什么容易错

慢思考

- 做法：不是一次写完，而是把推理当成一个“过程系统”：Search (搜索) —— Evaluate (评估) —— Revise (修正)
- 更像人的“慢思考”：把时间花在检查、比较、改错上，所以通常更稳

慢思考的本质

以 o1 系列为例

- 越来越多“推理型模型/推理型系统”会显式强调：**推理过程更强**（更擅长多步问题）+ **更鲁棒**（不那么容易被提示扰动搞崩）
- 和“慢思考范式”一致：推理阶段**更高的计算预算**+ 更强的**过程监督与验证机制**

代表性趋势

CoT 基础——为什么“写出步骤”会更好

CoT 是什么

把“隐式推理”外显成“中间状态序列”。在回答前/回答过程中，先输出若干**中间推理步骤**（中间结论、变量、子问题结果），再得到最终答案

为什么 CoT 往往能提升正确率？

- **分解**：CoT 强迫模型把问题拆成子问题
- **中间状态**：中间步骤相当于给模型一个“草稿纸”，减少一次性把所有约束塞进同一步的负担
- **对齐训练分布**：许多推理数据/解题文本本身就是“步骤式”。当模型按步骤输出时，更接近它在训练中学到的“解题范式”，因此更稳定

CoT 基础

关键现实

- CoT 不等于“真实推理”。
- CoT 是模型生成的文本，它可能只是“让答案看起来合理”的解释，而不一定是模型真正导致答案的原因。
 - 伪推理+蒙对

正确用法

- CoT = 草稿 (draft)
- 必须配套：自检/验证器+多路径一致性+工具校验
 - “CoT 提供可审计的中间步骤，但审计要靠验证机制完成。

维度	CoT 的收益 (Why it helps)	CoT 的风险 (What can go wrong)	课堂/工程对策 (Mitigation)
正确率	强制分解多步任务，减少跳步与遗漏	步骤变长后，误差可能累积	分段检查关键中间量；必要时短链化
可解释性	提供“中间状态”，方便人类或系统检查	解释不一定忠实：可能是“事后编造”	把 CoT 当草稿；加 verifier/规则/工具
可控性	可以注入格式与约束 (Plan→Solve→Check)	对提示敏感：同题不同写法差异大	结构化提示 + 自动化 prompt 优化
可审计性	可输出步骤、引用、计算过程	可能出现伪推理：看起来合理但不对	多路径一致性 + 工具校验 (PoT 等)
成本	对难题收益明显 (尤其多步)	token/时延成本增加	预算控制：少量采样 + 早停策略

结构化 CoT

A

Plan-and-Solve: 先列计划, 再逐步执行

- Plan-and-Solve 是什么: 模型先输出一个“解题计划/步骤清单”, 然后严格按计划逐步求解
- 为什么有效: 计划阶段强迫模型先识别子任务与依赖关系(先抽条件, 再列式/推导, 再计算, 再检查)
- 适用场景: 多步骤数学题、证明题的“结构梳理”、长文问答、需要引用证据的推理(先检索再推理)、需要输出固定格式(JSON/表格)的任务
- 常见失败模式与对策:
 - 失败 1: 计划写得很好, 但执行不按计划
 - 失败 2: 计划本身漏关键步骤
 - 失败 3: 计划过长导致成本高

Plan-and-Solve: 先确定路线, 再按路线执行, 减少跳步

B

Least-to-Most: 先解最简单子问题, 逐步递进

- Least-to-Most 是什么: 先让模型解决一个“最容易、最确定”的子问题, 再把得到的结果当作上下文, 递归/逐步解决更难的部分, 直到完成全题
- 为什么有效: 从确定性起步-逐步累积约束-对多约束任务更稳
- 适用场景: 多约束逻辑题、跨段文本推理(先抽实体与关系、再做推断)、复杂数学应用题(先抽变量含义与单位, 再列式)
- 常见失败模式与对策:
 - 失败 1: 子问题划分不合理失败
 - 失败 2: 早期子结论错了, 后面全错
 - 失败 3: 递进过程中上下文太长

VS

Least-to-Most: 先从确定性最强的子问题出发, 逐步收缩推理空间, 适合多约束任务

推理变搜索：ToT (Tree of Thoughts) 与多路径探索

ToT是什么



- ToT (树)：同一个问题同时探索多个候选思路（分支），每走几步就评估质量，不行就剪枝/回溯，再探索别的分支
- ToT = 多分支推理 + 分步评估 + 回溯选择

ToT 的核心组件

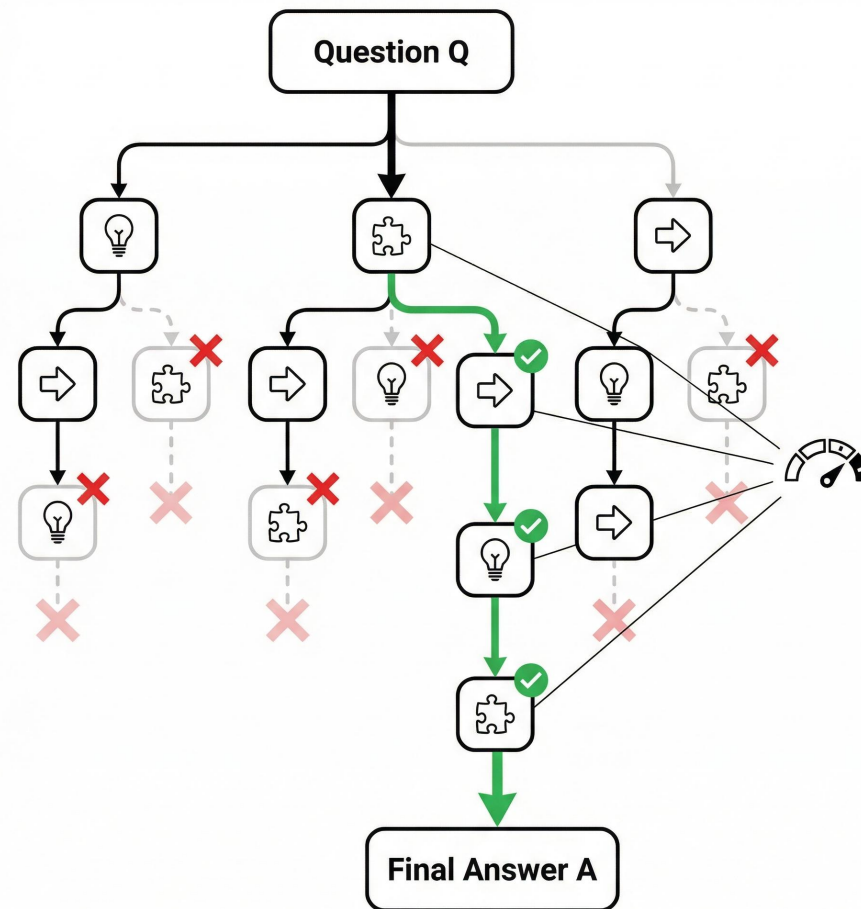


- Generate (生成候选 thought)：在某个“中间状态”下，模型生成若干个下一步候选（例如 3-10 个）
- Evaluate (自评估/外部评估)：用评分函数给每个候选打分
- Search/Backtrack (搜索与回溯)：保留 top-k 分支继续展开；低分分支剪掉；必要时回溯到上一个节点换路走

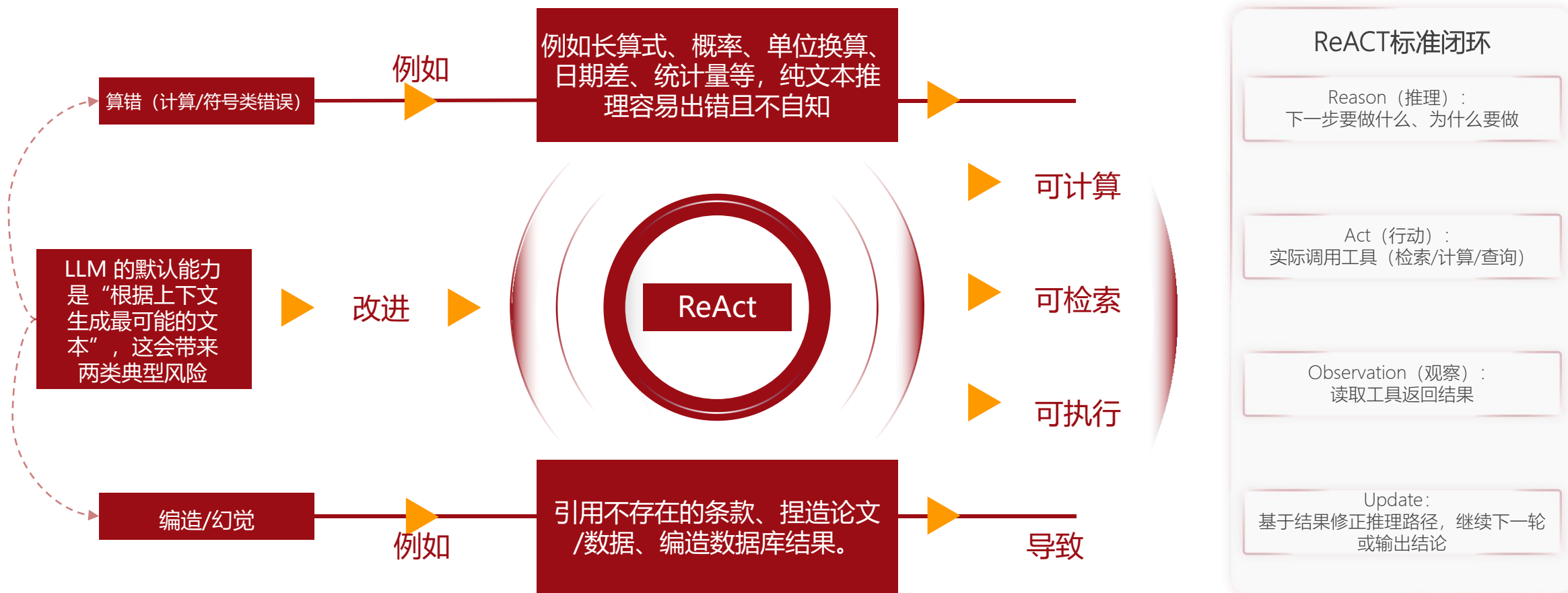
为什么 ToT 往往更强



避免单一路径早期错误的灾难性影响、把“推理”变成“优化问题”、天然支持回溯与纠错



推理与行动结合：工具调用 / ReAct 思路



直觉

像人在解题时“想一想 → 查一查/算一算 → 看结果 → 再想一想”

Prompt 自动化

痛点

手写 prompt 很**依赖经验**，换任务/换模型/换数据就容易失效；而且多模块系统（检索→抽取→推理→汇总）里，prompt 之间还会**相互影响**

把 prompt 当成“可被系统优化的参数”，用数据和指标把它自动调到更好

APE步骤

- 生成一堆候选指令
- 用一个评分函数评测每条指令
- 选择分数最高的指令

APE优缺点

- 优点：不靠灵感、快速迁移、指标驱动
- 缺点：指标偏了，优化就偏、容易“刷分”：在小验证集上会出现“过拟合某些题型/模板”

APE

DSPy

DSPy步骤

- Bootstrap 示例候选：从训练样本中构造/筛选可能有效的 few-shot 示例
- Propose 多种指令：提出多条 instruction 候选
- Search 最优组合：寻找“哪条指令 + 哪组示例”在指标上最好

DSPy优缺点

- 优点：自动优化 Prompt、适合多模块系统（尤其 RAG / 抽取+推理 / Agent pipeline）
- 缺点：高度依赖“你定义的指标”和“开发集分布”、优化成本不低（时间/Token/算力）、不适合“无监督、无指标、无数据”的任务



数学能力与可验证推理

LLM 做数学的三类典型失败

算术错误

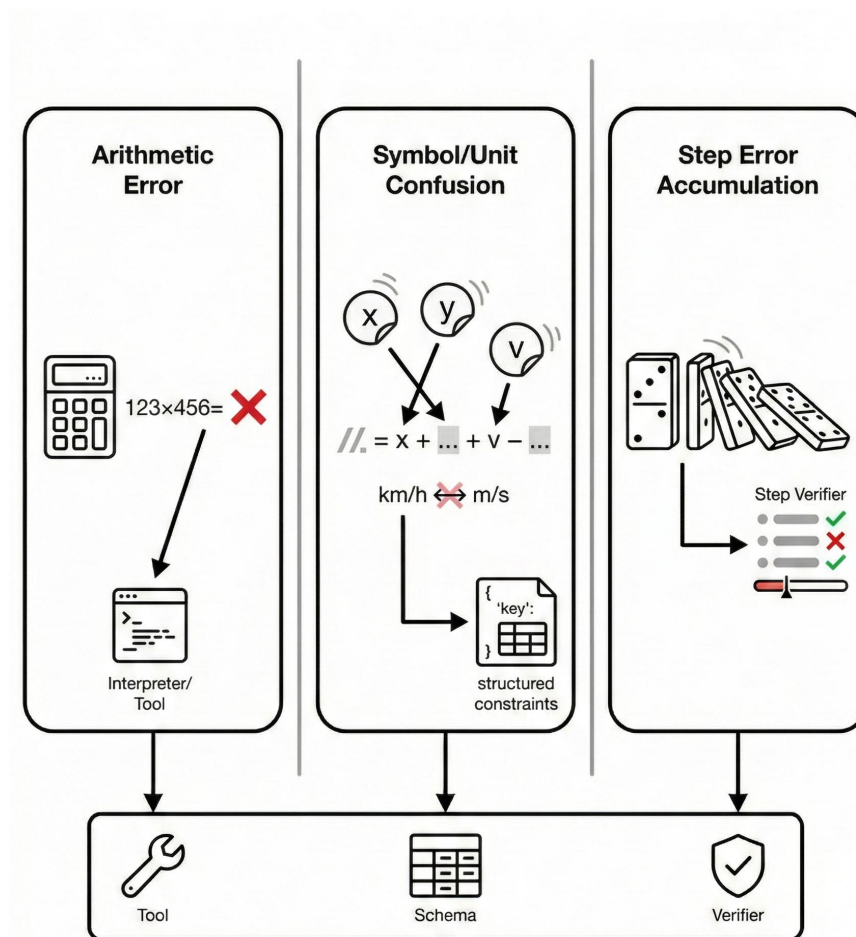
- 乘除、分数、小数、长算式、统计计算、日期差等“纯计算”环节出错
- 解决措施：把计算外包给解释器/工具（计算器、Python、CAS）

符号/单位混淆

- 变量定义不一致、符号偷换、单位未统一导致“看似正确、实际错误”
- 把 v 当速度又当体积、把 km/h 与 m/s 混用

多步链条 误差累积

- 推理链越长，越容易出现“早期一步错→后面全错”的多米诺效应加强于细胞行业的监管
- 解决措施：引入过程监督/验证器



PoT: Program-of-Thoughts



LYCHEE



哈尔滨工业大学(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

Program-of-Thoughts (PoT) 要求 LLM 在遇到需要精确计算的任务时, 先生成一段可执行的程序 (通常是 Python 代码或计算表达式), 然后把计算交给解释器/计算器执行, 再把执行结果组织成最终答案

PoT 是什么

让模型“写程序”而不是
“在文字里算”

- 代码能算得很准, 但算的是错公式
- 这是 PoT 无法自动解决的“语义理解”问题
- 变量名不一致、括号错误、漏单位转换、使用错误的公式、浮点/取整问题
- 或者生成不可运行代码 (语法错误)

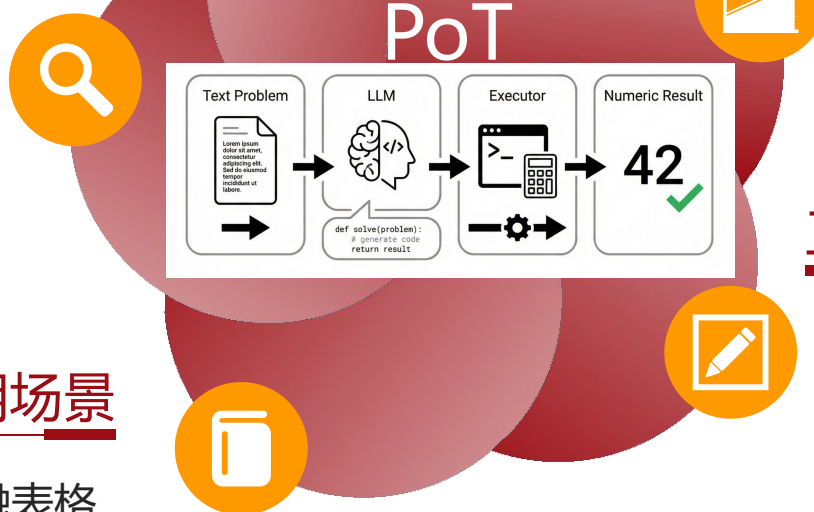
PoT 的局限

- 建模错 (理解题意/列式错)
- 代码生成错 (实现错/边界错)

为什么 PoT 有效

把不擅长的环节交给擅长的工具

- 算术准确率显著提升。复杂算式交给 Python/计算器, 不再靠模型“心算”。
- 推理可验证。程序本身可以检查: 变量是否定义、公式是否正确、结果是否一致。
- 便于审计与复现。同样输入得到同样结果; 便于在垂直领域做合规/财务核算。



适用场景

数学应用题、金融表格
计算、单位换算

工程对策让 PoT 更可靠

- 强制分两段输出: Program: (只放可执行代码) + Result: (只放执行输出与最终答案)
- 加断言/单元测试: 单位检查、范围检查
- 执行失败的回退策略: 语法错误就让模型“修复代码再运行” + 多次尝试仍失败则回退到“简化表达式 + 计算器”
- 对外部工具依赖的约束

过程监督/验证器—PRM

背景

为什么“只看最终答案”不够用：

- 多步推理任务（尤其数学/证明/复杂约束）里，错误往往发生在**中间某一步**
- 如果监督信号只在最后给一个“对/错”，**模型无法知道**

Outcome RM

输入：
题目 + 完整解答

输出：
一个整体分数（对/错或0~1）

优点：
标注简单、速度快

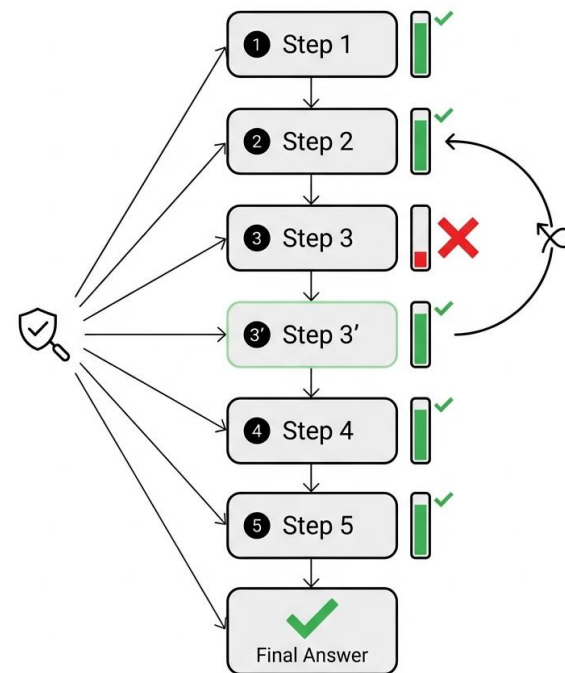
Process RM

输入：
题目 + 解答的前缀（到第 k 步为止）

输出：
对“当前这一步/这一段是否正确、是否走在正确轨道上”的分数

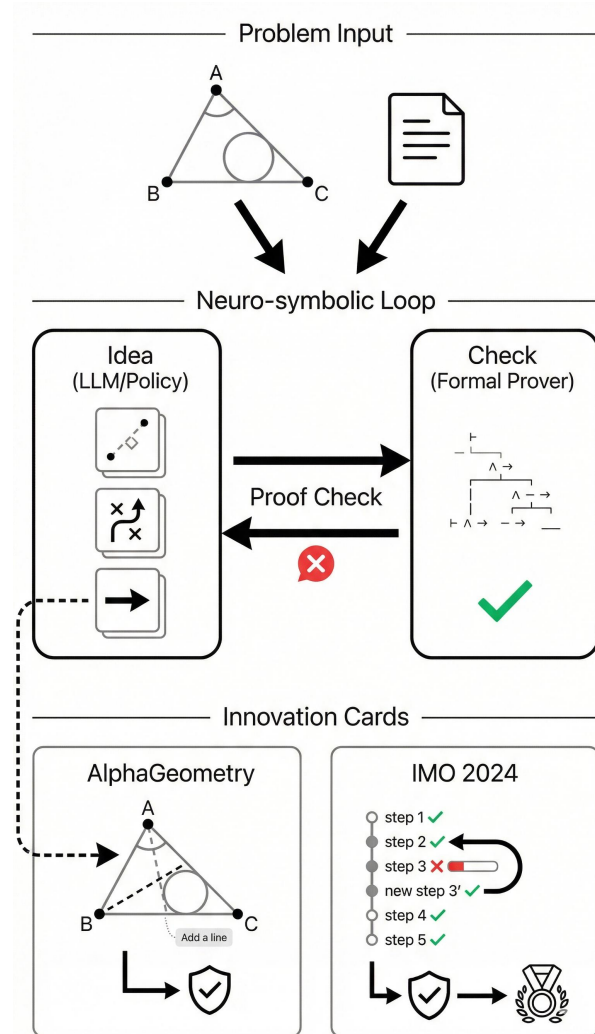
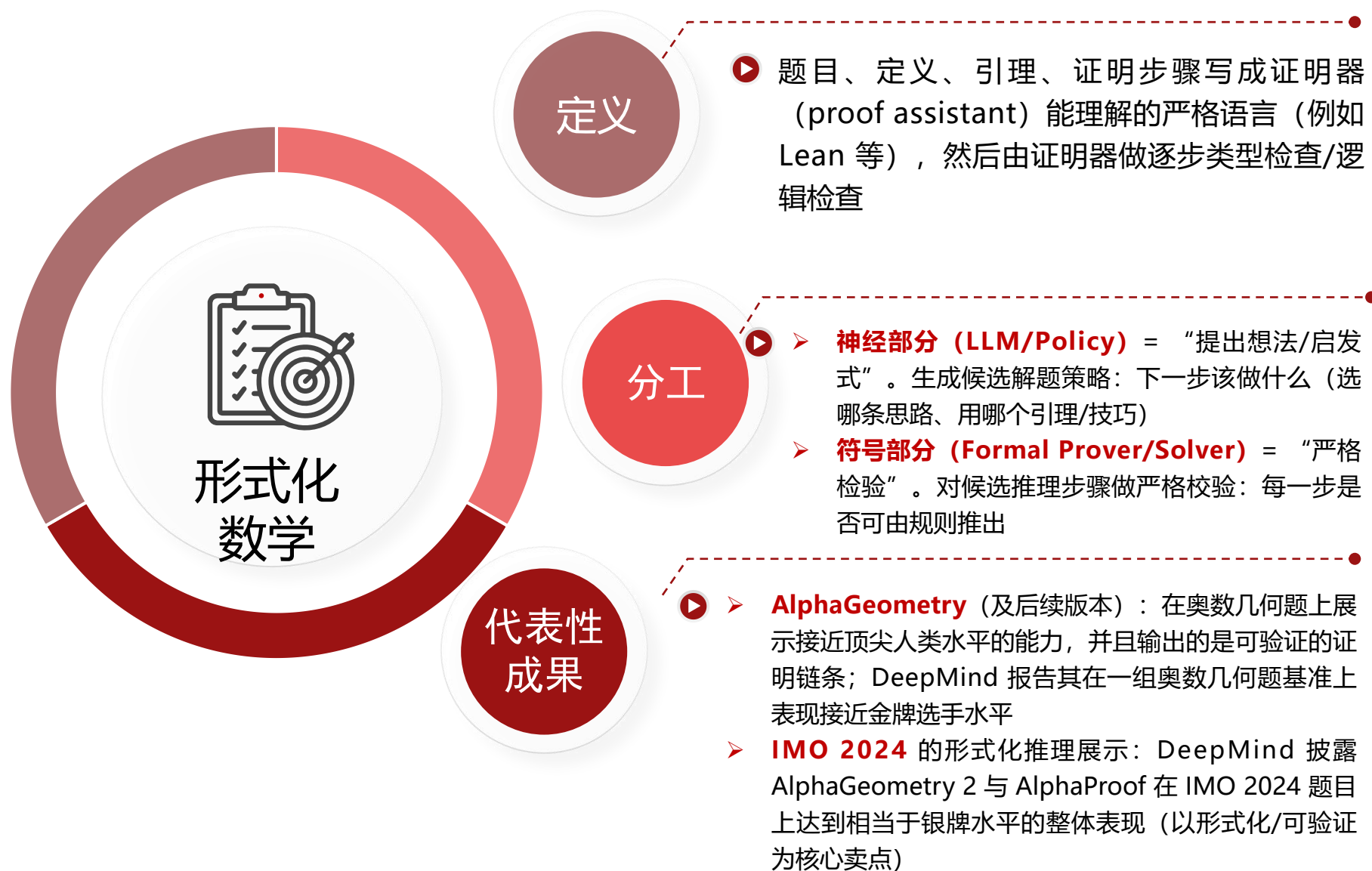
优点：
信号细，能在中途就发现偏离；对长链更有效

两种奖励模型的区别



PPM迭代步骤

形式化数学：AlphaGeometry / AlphaProof



数学推理的“现实问题清单”



LYCHEE



哈尔滨工业大学(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

评测集泄漏会夸大能力

- 很多数学题（尤其经典题库、竞赛题、公开数据集）在互联网广泛传播，模型训练数据可能间接包含题目或答案
- 结果：模型看起来“会推理”，但其实是“见过/背过”，评测被高估

数据污染 / 背题风险

数字/表述变化后性能下降

- 同一道题换个数字、换种表述（同义改写、顺序改变、加入无关信息），模型性能可能明显下降
- 数字换成更大/更怪的数就容易算错
- 把题干改写成更口语/更冗长，理解与建模错误增多

泛化问题

更强推理通常更慢、更贵

- CoT、ToT、多采样、自一致性、PRM 驱动的回滚重采样，都在显著增加 token 与调用次数
- “推理能力提升”常常来自“测试时算力/采样预算”的增加，而不是免费的能力增长

成本

数学推理的“现实问题清单”

方法	数据污染/背题风险	泛化脆弱（数字/改写）	成本（token/时延）	忠实性/可解释性风险	主要缓解手段
CoT（Chain-of-Thought）	中：题型模板易被记忆化	中-高：改写/换数易漂移	中：输出变长	高：可能“看似推理”但不忠实	加验证器/工具；结构化步骤；多样化评测
ToT（Tree of Thoughts）	中：搜索也可能走向背题模式	中：更稳但依赖评估器	高：分支+评估开销大	中：过程更像搜索，但评估器可能被糊弄	控制分支/深度；硬验证（规则/工具）；早停
PoT（Program-of-Thoughts）	低-中：结果可复算，背题优势小	中：建模仍可能随改写出错	中：生成代码+执行	低-中：可执行更可审计，但“算对公式”仍可能错	断言/单元测试；单位/变量表；执行失败回退
PRM（Process Reward / Verifier）	中：若训练数据污染会影响评分	中：可提升稳定性但可能过拟合评测	高：逐步打分+回滚重采样	低-中：更接近“可检查”，但评分器也会错	多指标与硬校验；holdout 新题；限制回滚次数
Formal（形式化证明/求解器）	低：形式系统约束强，背题空间小	中：受形式化覆盖范围限制	高：形式化搜索/验证可能很慢	低：通过即为“可机检正确”	改进形式化库；神经提案+符号验证；缓存与剪枝



垂直领域推理 & 抽取+推理

垂直领域推理的目标：可靠、可审计、可复现



垂直领域

为什么垂直领域不能“差不多”

在法律、医疗、金融、企业合规等场景里，LLM 输出的风险不是“答错一题”，而是：

法律：误引条款、错误归责 → 合规与诉讼风险

医疗：误判剂量/禁忌 → 安全风险

金融：算错利率/口径 → 决策与监管风险

企业合规：错误解读制度 → 审计不通过、处罚

三个硬指标

- 可追溯 (Traceable)：结论必须能“指回证据”
- 可验证 (Verifiable)：关键步骤必须能“复核”
- 可控 (Controllable)：输出要稳定、结构要可控、边界要清楚

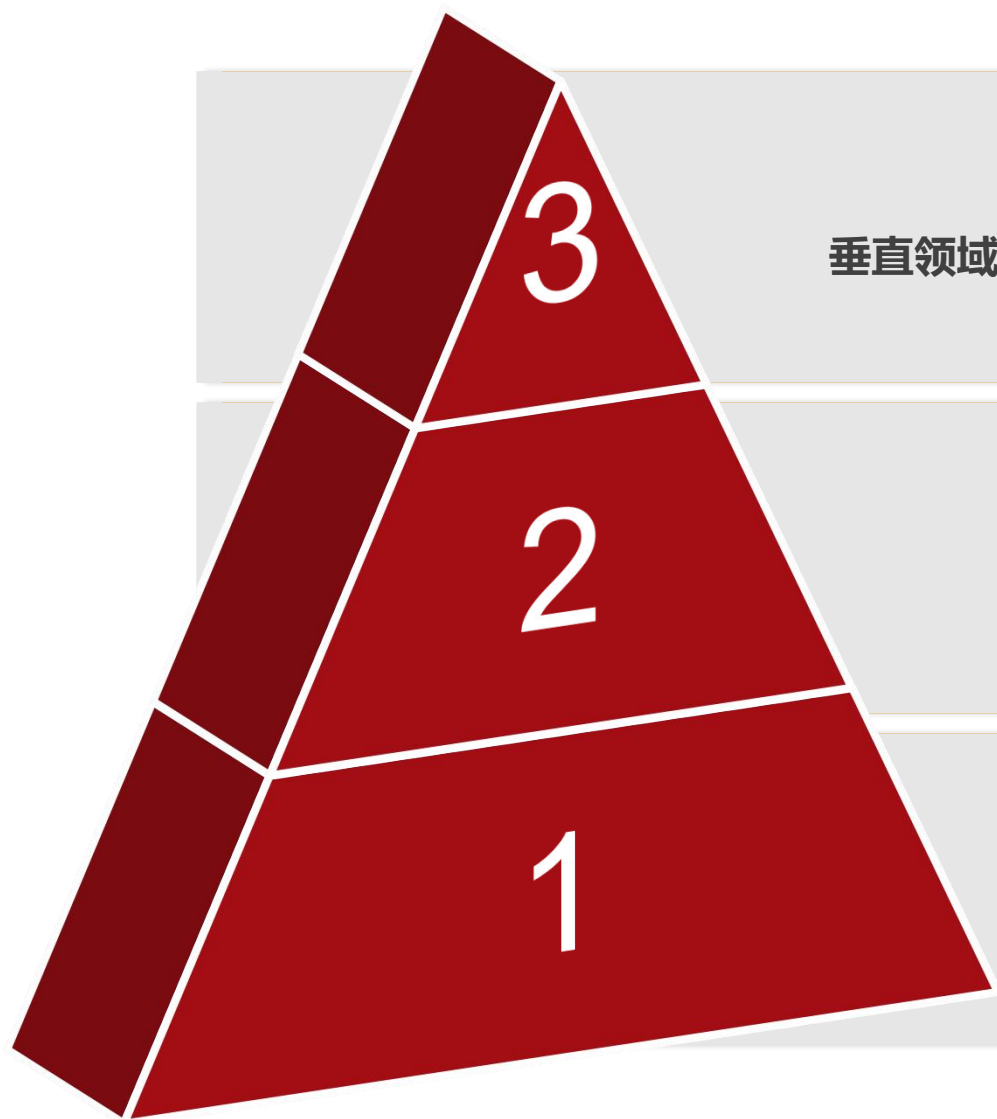
1.在法律、医疗、金融与企业合规等高风险场景，模型能力不以“回答像不像”衡量，而以“**能否被审计**”衡量。

2.可用的垂直领域推理系统**必须同时满足三件事**：结论可追溯到具体证据 (Traceable)，关键判断可通过规则/计算复核 (Verifiable)，输出结构与行为边界稳定可控 (Controllable)。

3.只有把推理纳入“证据链 + 校验链 + 结构化输出”的工程闭环，结果才具备**可靠上线**与**可复现复盘**的条件。



抽取 + 推理 (Extract-then-Reason)



垂直领域效果显著

3.Result ▶▶▶

LLM 负责“读懂并抽字段 + 解释输出”，核心判断交给“可验证的推理层”

2.Reason ▶▶▶

在结构化对象上做“硬推理”
规则引擎/阈值判断 (if-then)、SQL 聚合/过滤、
数学计算/口径复核、最后让 LLM 做解释与汇总

1.Extract ▶▶▶

把“自由生成”换成“先变成可计算对象”
比如把非结构化文本（合同、病历、财报、制度）转
换成结构化对象

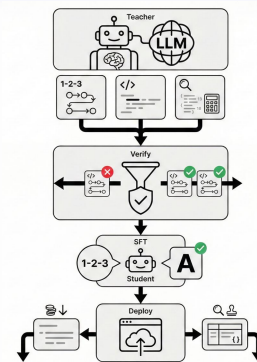
推理蒸馏：把强模型“会推理的轨迹”迁移到小模型

推理蒸馏

让强模型 (Teacher) 把 “怎么推理” 的过程写出来 (分步解释/计算程序/中间结论)，再用这些 “推理轨迹 (reasoning traces)” 去监督微调小模型 (Student)，把推理能力迁移过去

- ✓ 强模型推理强但贵、慢、难部署；小模型便宜快但推理弱
- ✓ 推理蒸馏希望把 “推理增益” 从 test-time compute (推理时多采样/多步搜索) 转移到 train-time learning (训练时吸收老师的轨迹)，上线时用小模型也能更稳

- ✓ 生成轨迹
- ✓ 过滤与校验 (关键)
- ✓ 监督微调
- ✓ 上线推理



- ✓ **Distilling Step-by-Step**: 把 Teacher 的 “分步解释” 当成额外训练信号，小模型更容易学到推理结构，用更少数据也能更强
- ✓ **rationale / 过程蒸馏**: 泛称 “把中间推理文本/步骤” 作为监督，而不是只蒸馏最终答案
- ✓ **PaD (Program-aided Distillation)**: 把 “推理过程” 尽量换成可执行程序，用执行结果做校验/自我修正，减少 “错误 CoT” 污染学生

解决的现实问题

典型 workflow

代表性脉络

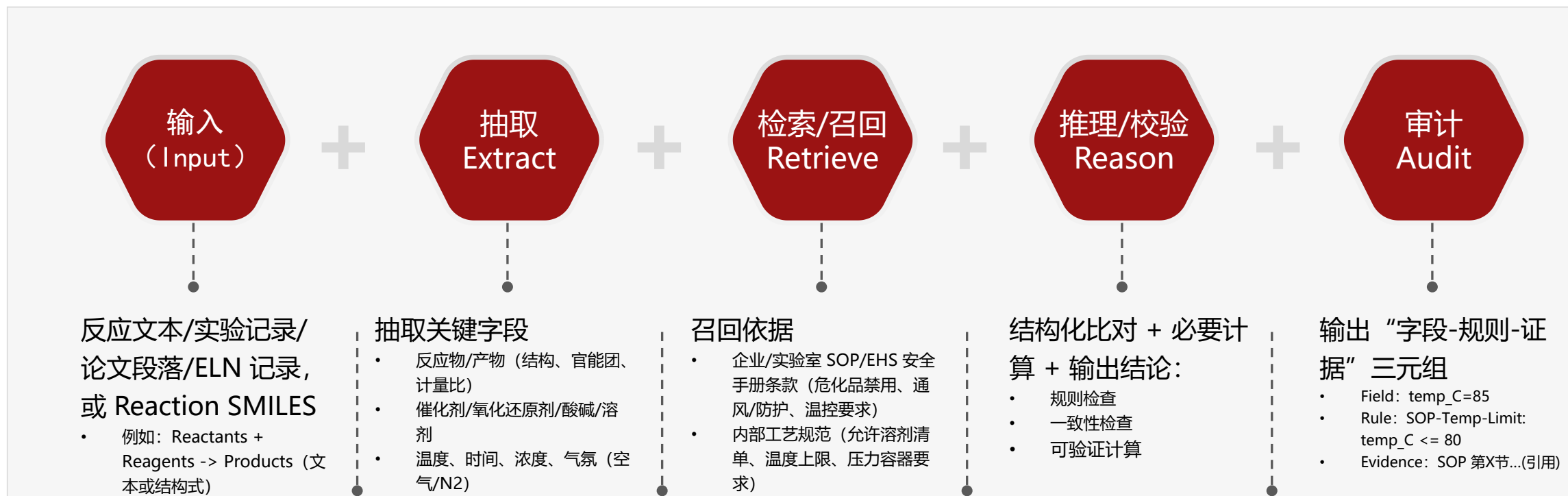
局限与风险：教师轨迹可能不忠实+偏差会被放大复制

端到端案例：反应可行性与条件合规审查



化学垂域推理：从反应描述到可审计结论

在化学场景中，“看起来合理”的回答不够用：系统必须把反应文本先结构化抽取为可计算字段，检索并引用 SOP/EHS/工艺规范作为依据，再用规则与计算完成可验证的判定，最终输出可追溯的“字段-规则-证据”链条，确保结论可靠、可复核、可审计



01

Prompt 正在 “结构化 + 自动化优化” (APE/DSPy)

02

推理正在 “搜索化 + 验证化” (ToT/Verifier/PRM)

03

数学正在 “工具化 + 形式化” (PoT + Formal Proof)